МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №1

по дисциплине «Построение и анализ алгоритмов»

ТЕМА: Поиск с возвратом

Вариант: 1р

Студент гр. 4343 _____ Гизатов Я.Р.

Преподаватель _____ Жангиров Т. Р.

Санкт-Петербург

2026

Задание

У Вовы много квадратных обрезков доски. Их стороны (размер) изменяются от 1 до $N - 1$, и у него есть неограниченное число обрезков любого размера. Но ему очень хочется получить большую столешницу – квадрат размера N. Он может получить ее, собрав из уже имеющихся обрезков(квадратов).

Например, столешница размера 7×7 может быть построена из 9 обрезков (рисунок 1).

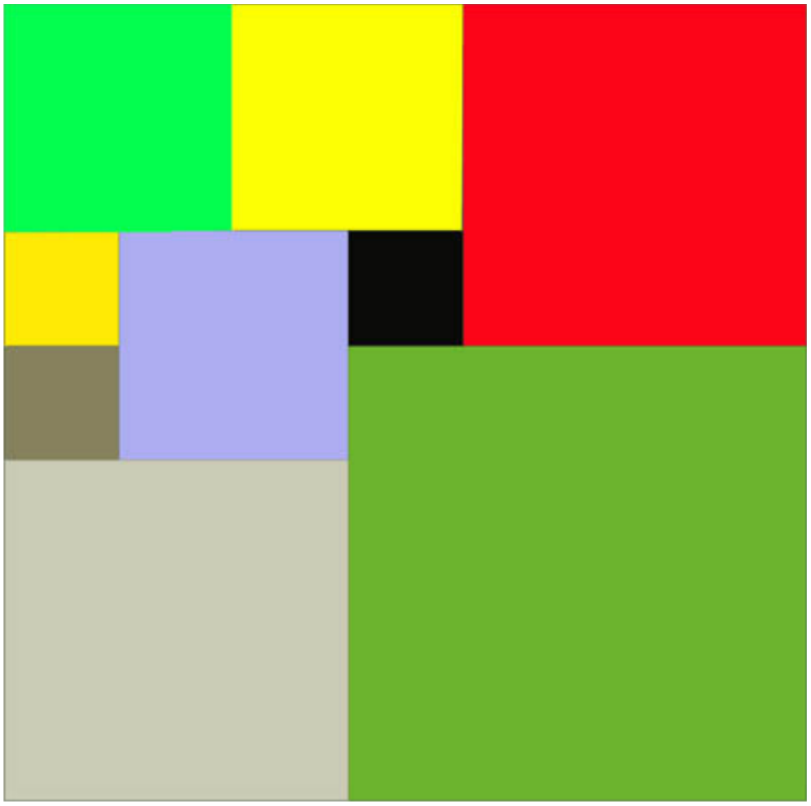

Рисунок 1 – Заполнение квадрата $N = 7$

Внутри столешницы не должно быть пустот, обрезки не должны выходить за пределы столешницы и не должны перекрываться. Кроме того, Вова хочет использовать минимально возможное число обрезков.

Входные данные

Размер столешницы - одно целое число N $(2 \leq N \leq 20)$.

Выходные данные

Одно число K, задающее минимальное количество обрезков(квадратов), из которых можно построить

столешницу(квадрат) заданного размера N. Далее должны идти K строк, каждая из которых должна содержать три целых числа x, y и w, задающие координаты левого верхнего угла $(1 \leq x, y \leq N)$ и длину стороны соответствующего обрезка(квадрата).

Пример входных данных

7

Соответствующие выходные данные

9

1 1 2

1 3 2

3 1 1

4 1 1

3 2 2

5 1 3

4 4 4

1 5 3

3 4 1

Вариант

Вар. 1р. Рекурсивный бэктрекинг. Выполнение на Stepik всех трёх заданий в разделе 2.

Задание 2

Условие задачи такое же, как и в предыдущем степе, но размер столешницы стал больше – $2 \leq N \leq 30$.

Задание 3

Условие задачи такое же, как и в предыдущем степе, но размер столешницы стал больше – $2 \leq N \leq 40$.

Выполнение работы

Программа решает задачу поиска минимального количества квадратов, которыми можно замостить квадратную столешницу размера $N \times N$, при условии, что ни один из используемых квадратов не может иметь сторону, равную N (то есть максимальный размер квадрата ограничен N-1). Задача является частным случаем проблемы упаковки квадратов и решается методом рекурсивного перебора с возвратом (backtracking) с использованием ряда оптимизаций.

Основные структуры данных

rows (vector<unsigned long long>)

Вектор битовых масок размером N, где каждый элемент представляет собой 64-битное число, кодирующее состояние занятости соответствующей строки столешницы. Младшие N битов каждого числа используются для хранения информации о клетках: если бит в позиции X установлен в 1, клетка с координатами (X, Y) занята, если 0 – свободна.

Square (tuple<int,int,int>)

Кортеж для хранения параметров одного установленного квадрата, где первый элемент хранит координату X левого верхнего угла (значение от 1 до N), второй элемент – координату Y левого верхнего угла, третий элемент – размер стороны квадрата.

current (vector<tuple<int,int,int>>)

Вектор, хранящий список всех квадратов, установленных на текущем шаге поиска. Поддерживается синхронно с изменениями в битовых масках rows: при добавлении квадрата соответствующий кортеж добавляется в вектор, при удалении убирается последний элемент.

best_solution (vector<tuple<int,int,int>>)

Глобальная переменная, хранящая лучшее найденное на данный момент решение в виде вектора кортежей с параметрами квадратов. Обновляется при обнаружении покрытия с меньшим количеством квадратов, чем текущий рекорд.

best_count (int)

Целочисленная переменная, хранящая количество квадратов в лучшем найденном решении. Инициализируется значением общей площади N*N (тривиальное решение из единичных квадратов) и уменьшается по мере нахождения более качественных покрытий.

Описание алгоритма

Алгоритм программы представляет собой рекурсивный поиск с возвратом (backtracking) для нахождения минимального количества квадратов, которыми можно замостить квадратную столешницу размера $N \times N$, с ограничением, что ни один квадрат не может иметь сторону, равную N. Работа начинается с проверки особых случаев в функции easy_way: если N чётное, кратное 3 или кратное 5, программа сразу выдаёт известное оптимальное решение, разбивающее столешницу на 4, 6 или 8 квадратов соответственно, полностью избегая трудоёмкого перебора. Если N не попадает в эти категории, алгоритм переходит к основной процедуре поиска.

Перед запуском рекурсивного поиска выполняется эвристическое начальное размещение трёх квадратов в левом верхнем углу столешницы. Вычисляется значение first_size = int(max_size / 2) + 1, где max_size = N-1. В позицию (1,1) помещается квадрат со стороной first_size, а в позиции (first_size+1, 1) и (1, first_size+1) - два квадрата со стороной first_size-1. Это размещение занимает значительную площадь и основано на известных закономерностях оптимальных разбиений, что позволяет сократить глубину последующего перебора. Соответствующим образом обновляются битовые маски в векторе rows и список текущих квадратов current.

Далее запускается итеративный процесс с увеличением ограничения на количество квадратов. Переменная best_count инициализируется значением общей площади N×N, что соответствует тривиальному решению из единичных квадратов. Нижняя оценка lower устанавливается равной количеству уже размещённых квадратов плюс один. Для каждого значения

limit, начиная с lower и увеличивая, пока не будет найдено решение, вызывается рекурсивная функция backtrack. Как только для очередного limit функция возвращает true, алгоритм останавливается, гарантируя оптимальность найденного решения.

Описание рекурсивной функции

bool backtrack(int count, int area_used, int limit, ofstream& file) – рекурсивная функция, которая получает текущее количество квадратов count, использованную площадь area_used, ограничение limit и ссылку на файл для логирования. В начале работы функция проверяет, не превышает ли текущее количество квадратов установленный лимит если да, поиск по данной ветви прекращается. Затем с помощью вспомогательной функции first_free определяется первая свободная клетка в порядке обхода сверху вниз и слева направо. Если свободных клеток нет, достигнуто полное покрытие: сравнивается текущее количество квадратов с глобальным рекордом best_count, и при улучшении обновляются best_solution и best_count, после чего функция возвращает true. Если свободная клетка существует, перебирается максимально возможный размер квадрата, который можно в неё поместить, то он ограничивается расстояниями до правого и нижнего краёв, а также глобальным ограничением max_size.

Оптимизации алгоритма

Отсечение по верхней границе: Функция backtrack получает параметр limit, который задаёт максимально допустимое количество квадратов в искомом решении. В начале выполнения функции проверяется условие if (count > limit) return false; если текущее количество уже уложенных квадратов превышает установленный лимит, дальнейший поиск по данной ветви немедленно прекращается, так как эта ветвь заведомо не может привести к решению, лучшему чем уже найденное.

Итеративное углубление: Поиск оптимального решения выполняется не с фиксированным ограничением, а последовательно для возрастающих значений limit, начиная с нижней оценки squares_placed + 1 и

увеличивая на единицу, пока не будет найдено первое полное покрытие. Как только для очередного limit функция backtrack возвращает true, алгоритм останавливается, что гарантирует минимальность количества квадратов в найденном решении.

Эвристическое начальное размещение: Вместо старта с пустой доской программа сразу укладывает три квадрата в левом верхнем углу: квадрат со стороной first_size = int(max_size/2) + 1 в позиции (1,1) и два квадрата со стороной first_size-1 справа и снизу от него в позициях (first_size+1, 1) и (1, first_size+1). Это позволяет занять значительную площадь до начала перебора и сокращает глубину рекурсии.

Перебор размеров от большего к меньшему: Для каждой найденной свободной клетки в цикле перебираются возможные размеры квадратов, начиная с максимально допустимого (ограниченного расстоянием до краёв доски и значением max_size) и уменьшая до 1. Такой порядок увеличивает вероятность быстрого заполнения оставшейся площади и достижения полного покрытия с меньшим числом квадратов, что позволяет раньше обновить рекорд и усилить отсечение.

Битовые маски для представления строк: Состояние столешницы хранится в векторе rows из 64-битных целых чисел, где каждый бит соответствует одной клетке строки. Проверка can_place выполняется путём наложения маски ((1ull << s) - 1) << x на соответствующие строки и проверки rows[i] & mask за O(s) операций. Установка и удаление квадрата выполняются через битовые операции |= и &= ~ соответственно, что значительно быстрее поэлементной работы с двумерным массивом.

Выбор первой свободной клетки: Функция first_free всегда находит самую верхнюю и самую левую свободную клетку в порядке обхода строк сверху вниз и столбцов слева направо. Фиксированный порядок исключает рассмотрение симметричных вариантов размещения и уменьшает общее количество проверяемых комбинаций.

Предварительная обработка частных случаев: Для размеров N, которые делятся на 2, 3 или 5 (кроме самих этих чисел), в функции easy_way реализованы готовые аналитические разбиения на 4, 6 или 8 квадратов соответственно. При выполнении любого из этих условий программа сразу выводит соответствующее решение и завершает работу, полностью избегая рекурсивного перебора.

Оценка сложности

Задача сводится к поиску оптимального случая расстановок, только для квадратов, стороны которых являются простыми числами. Поэтому первый цикл for, который перебирает количество квадратов от 3 до N^2, дает нам сложность не больше N, так как число квадратов даже для самого большого числа и близко не подходит к самому числу (например, у квадрата со стороной 37, лучшая расстановка займет 15 квадратов).

Далее в рекурсивной функции находится цикл перебора размера квадрата от большего $N/2$ до наименьшего 1. И в случае возможности поставить квадрат запускается рекурсия, ищущая новое место для нового квадрата.

Данное действие можно обозначить за число сочетаний с повторением, так как алгоритм рассматривает все варианты и при этом всегда пытается установить квадрат в верхнюю левую клетку.

Для удобства обозначим $\frac{n}{2} = k$

Тогда при первой итерации $limit = 4$ и будет перебор 1 квадрата из k возможных, после чего рекурсия остановится, число этих итераций будет равно:

$$\bar{C}_k^1 = C_{k+1-1}^1 = C_k^1$$

Далее $limit = 5$ и перебор будет искать число сочетаний с повтором из 2 по k

$$\bar{C}_k^2 = C_{k+2-1}^1 = C_{k+1}^1$$

И если бы алгоритм не останавливался за счет оптимизаций, он бы дошел до $\bar{C}_k^k = C_{k+k-1}^k = C_{2k-1}^k$

У нас получился ряд

$\sum_{i=1}^{k} C_{k+i-1}^i$

Оценим его сверху и приведем ряд по биному ньютона к 2^n

$\sum_{i=1}^{k} C_{k+i-1}^i < \sum_{i=1}^{2k} C_{2k}^i = 2^{2k} = 2^n$

Так как мы оценили сверху то оценка алгоритма будет $O(2^n)$ ограничивая сложность сверху, что и доказывает график на рисунке 2.

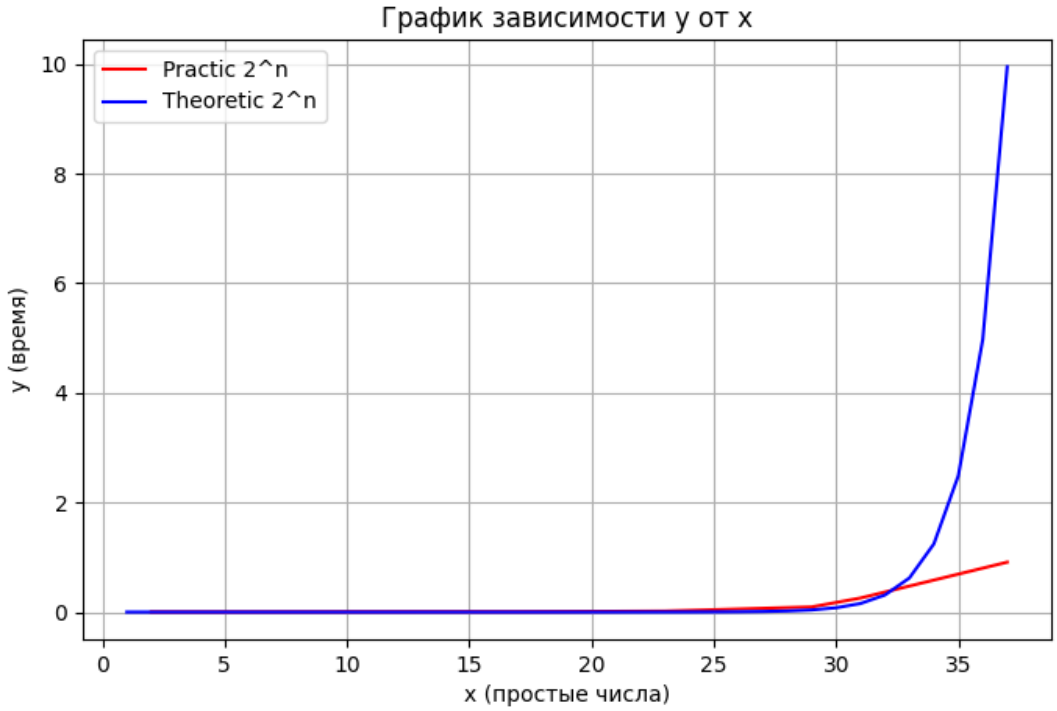

Рисунок 2 – График оценки алгоритма

Заметим, что практическая оценка выполняется быстрее чем теоретическая, из-за оптимизаций алгоритма.

Оценка по памяти

Так как заполнение квадрата хранится в векторе с unsigned long long значениями, а их всего N, а также лучшее решение и текущее решение будут иметь не более N кортежей, то оценка памяти сводится к $O(n)$.

Тестирование

1. Проверка для простых чисел $N = 2$

```
2
4
1 1 1
2 1 1
1 2 1
2 2 1
Elapsed time: 0.000999 seconds.
```

2. Проверка для простых чисел $N = 3$

```
3
6
1 1 2
3 1 1
1 3 1
3 2 1
2 3 1
3 3 1
Elapsed time: 0.002 seconds.
```

3. Проверка для простых чисел $N = 5$

```
5
8
1 1 3
4 1 2
1 4 2
4 3 2
3 4 1
3 5 1
4 5 1
5 5 1
Elapsed time: 0.002999 seconds.
```

4. Проверка для числа, делящегося на 2, $N = 20$

```
20
4
1 1 10
11 1 10
1 11 10
11 11 10
```

5. Проверка для числа, делящегося на 3, $N = 33$

```
33
6
1 1 22
23 1 11
1 23 11
23 23 11
23 12 11
12 23 11
```

6. Проверка для числа, делящегося на 5, $N = 35$

```
35
8
1 1 21
22 1 14
1 22 14
22 22 14
```

```
22 15 7
29 15 7
15 22 7
15 29 7
```

7. Проверка для простого числа наибольшего до 40, $N = 37$

```
37
15
1 1 19
20 1 18
1 20 18
20 19 2
22 19 5
27 19 11
19 20 1
19 21 3
19 24 8
27 30 3
30 30 8
19 32 6
25 32 1
26 32 1
25 33 5
Elapsed time: 0.874 seconds.
```

Программа для всех случаев отработала корректно.

Вывод

В ходе выполнения лабораторной работы был реализован алгоритм поиска с возвратом для решения задачи упаковки квадратов — нахождения минимального количества обрезков размерами от 1 до $N - 1$, необходимых для замощения квадратной столешницы $N \times N$. Программа успешно обрабатывает входные данные в диапазоне от 2 до 40, выдавая оптимальное количество квадратов и их координаты. Для повышения эффективности использован ряд оптимизаций: битовые маски для хранения состояния строк, отсечение по верхней границе, итеративное углубление, эвристическое начальное размещение трёх квадратов, перебор размеров от большего к меньшему и предварительная обработка частных случаев для N, кратных 2, 3 и 5. Проведённые теоретические оценки показали, что верхняя граница сложности алгоритма составляет $O(2^n)$, однако благодаря применённым оптимизациям практическое время выполнения значительно ниже, что подтверждается построенным графиком зависимости времени от размера N. Тестирование на наборе входных данных, включая простые числа до 37 и числа, кратные 2, 3 и 5, подтвердило корректность работы программы и достижение минимального количества квадратов для всех рассмотренных случаев. Таким образом, поставленная задача решена полностью, алгоритм демонстрирует приемлемую производительность для заданных ограничений.

Приложение

Код программы:

Main.cpp

```
#include <iostream>
#include <vector>
#include <tuple>
#include <algorithm>
#include <fstream>
#include <bitset>

#include <clocale>

#include <ctime>
#include <thread>

using namespace std;

int N;
int max_size;
int total_area;
vector<unsigned long long> rows;
vector<tuple<int,int,int>> best_solution;
vector<tuple<int,int,int>> current;
int best_count;
bool graph_log;


void  write_log(ofstream&  file,  const  vector<tuple<int,int,int>>&
squares) {
    vector<vector<string>> grid(N, vector<string>(N, "▒"));

    for (size_t idx = 0; idx < squares.size(); ++idx) {
        char c;
        if (idx < 26)
            c = 'A' + idx;
        else if (idx < 52)
            c = '1' + (idx - 26);
        else
            c = '?';

        int x = get<0>(squares[idx]) - 1;
        int y = get<1>(squares[idx]) - 1;
        int s = get<2>(squares[idx]);

        for (int i = y; i < y + s; ++i)
            for (int j = x; j < x + s; ++j)
                grid[i][j] = c;
    }
    for (int i = 0; i < N; ++i) {
        for (int j = 0; j < N; ++j)
            file << grid[i][j] << " ";
        file << endl;
    }
    file << endl;
}


bool can_place(int x, int y, int s) {
    if (x + s > N || y + s > N) return false;
    unsigned long long mask = ((1ull << s) - 1) << x;
```

```
        for (int i = y; i < y + s; ++i)
            if (rows[i] & mask) return false;
        return true;
    }


    void place(int x, int y, int s, bool set, ofstream& file) {
        unsigned long long mask = ((1ull << s) - 1) << x;
        for (int i = y; i < y + s; ++i) {
            if (set) rows[i] |= mask;
            else rows[i] &= ~mask;
        }

        if (graph_log) {
            if (set) {
                file << "Установка квадрата в координаты " << "x,y = " <<
x << "," << y << " со стороной N = " << s << endl;
                write_log(file, current);
            } else {
                file << "Удаление квадрата в координатах " << "x,y = " <<
x << "," << y << " со стороной N = " << s << endl;
                write_log(file, current);
            }
        }
    }


    bool first_free(int &x, int &y) {
        for (y = 0; y < N; ++y) {
            if (rows[y] != (1ull << N) - 1) {
                for (x = 0; x < N; ++x)
                    if (!((rows[y] >> x) & 1)) return true;
            }
        }
        return false;
    }


    bool backtrack(int count, int area_used, int limit, ofstream& file) {

        if (count > limit) return false;

        int x, y;
        if (!first_free(x, y)) {
            if (count < best_count) {
                best_count = count;
                best_solution = current;
            }
            return true;
        }

        int max_possible = min(N - x, N - y);
        max_possible = min(max_possible, max_size);

        for (int s = max_possible; s >= 1; --s) {
            if (count + 1 > limit) break;
            if (can_place(x, y, s)) {
                place(x, y, s, true, file);
                current.emplace_back(x + 1, y + 1, s);
                if (backtrack(count + 1, area_used + s * s, limit, file))
{
                    place(x, y, s, false, file);
                    current.pop_back();
                    return true;
```

```
                }
                place(x, y, s, false, file);
                current.pop_back();
            }
        }
        return false;
    }


    bool easy_way(int N){
        if (N != 2 and N % 2 == 0) {
            int half = N / 2;

            cout << 4 << "\n";
            for (int j = 0; j < 2; j++) {
                int y = 1 + j * half;

                for (int i = 0; i < 2; i++) {
                    int x = 1 + i * half;
                    cout << x << " " << y << " " << half << "\n";
                }
            }
            return true;
        } if (N != 3 and N % 3 == 0){
            int third = N / 3;
            int two_thirds = 2 * third;
            cout << 6 << "\n";

            cout << "1 1 " << two_thirds << "\n";
            cout << two_thirds + 1 << " 1 " << third << "\n";
            cout << "1 " << two_thirds + 1 << " " << third << "\n";
            cout << two_thirds + 1 << " " << two_thirds + 1 << " " << third
<< "\n";
            cout << two_thirds + 1 << " " << 1 + third << " " << third <<
"\n";
            cout << 1 + third << " " << two_thirds + 1 << " " << third <<
"\n";

            return true;

        } if (N != 5 and N % 5 == 0){
            int fift = N / 5;
            int two_fift = 2 * fift;
            int three_fift = 3 * fift;
            cout << 8 << "\n";

            cout << "1 1 " << three_fift << "\n";

            cout << three_fift + 1 << " 1 " << two_fift << "\n";
            cout << "1 " << three_fift + 1 << " " << two_fift << "\n";

            cout << three_fift + 1 << " " << three_fift + 1 << " " <<
two_fift << "\n";

            cout << three_fift + 1 << " " << two_fift + 1 << " " << fift
<< "\n";
            cout << three_fift + 1 + fift << " " << two_fift + 1 << " " <<
fift << "\n";

            cout << two_fift + 1 << " " << three_fift + 1 << " " << fift
<< "\n";
            cout << two_fift + 1 << " " << three_fift + 1 + fift << " " <<
fift << "\n";
```

```
            return true;
        }
        return false;
    }

    int main() {

        ofstream outFile("output.log");
        ofstream graph_file("times.txt");

        graph_log = false;
        bool time_log = false;

        vector<int> primes = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37};

        int time_cnt_iter = 1;
        if (time_log) time_cnt_iter = 12;
        else cin >> N;
        for(int i = 0; i < time_cnt_iter; i++){

            if (time_log) N = primes[i];
            auto start = std::chrono::high_resolution_clock::now();

            if(easy_way(N)) return 0;

            max_size = N - 1;
            total_area = N * N;
            best_count = total_area;

            int first_size = int(max_size / 2) + 1;

            int squares_placed = 0;

            rows.assign(N, 0);

            current.clear();

            current.emplace_back(1, 1, first_size);
            place(0, 0, first_size, true, outFile);

            current.emplace_back(first_size + 1, 1, first_size - 1);
            place(first_size, 0, first_size - 1, true, outFile);

            current.emplace_back(1, first_size + 1, first_size - 1);
            place(0, first_size, first_size - 1, true, outFile);


            squares_placed +=3;

            int used_area = first_size * first_size + 2 * ((first_size -
1)*(first_size - 1));

            int remaining = total_area - used_area;
            int lower = squares_placed + 1;


            for (int limit = lower; limit < best_count; ++limit) {
                if (backtrack(3, used_area, limit, outFile)) {
                    break;
                }
            }

            if (best_count == total_area) {
                cout << total_area << "\n";
```

```
                for (int y = 0; y < N; ++y)
                    for (int x = 0; x < N; ++x)
                        cout << x + 1 << " " << y + 1 << " 1\n";
            } else {
                cout << best_count << "\n";
                for (const auto& t : best_solution)
                    cout << get<0>(t) << " " << get<1>(t) << " " << get<2>(t)
<< "\n";
            }

            outFile << "Итоговый квадрат:" << endl;
            write_log(outFile, best_solution);

            auto end = std::chrono::high_resolution_clock::now();
            std::chrono::duration<double> duration = end - start;

            graph_file << N << " " << duration.count() << endl;

            std::cout << "Elapsed time: " << duration.count() << "
seconds." << std::endl;
        }

        return 0;
    }
```

Graph.py

```
import matplotlib.pyplot as plt
import numpy as np

x = []
y = []

with open('times.txt', 'r') as file:
    for line in file:
        if line.strip():
            parts = line.split()
            if len(parts) >= 2:
                x.append(float(parts[0]))
                y.append(float(parts[1]))



x_t = np.linspace(1, 37, 37)
y_t = list(map(lambda x: float(2**x), x_t))

k = 0

for i, elem in enumerate(x):
    print(i, elem)
    k += y_t[int(elem)-1]/y[i]

k = k / len(x)


y_t = list(map(lambda x: x/k, y_t))

plt.figure(figsize=(8, 5))
plt.plot(x, y, color='r', label='Practic 2^n')
plt.plot(x_t, y_t, color='b', label='Theoretic 2^n')

plt.xlabel('x (простые числа)')
plt.ylabel('y (время)')
plt.title('График зависимости y от x')
```

```
plt.grid(True)
plt.legend()

plt.show()
```